to your MySQL configuration file (*/etc/my.cnf* in CentOS). The variable *long_query_time* defines the threshold time beyond which queries will be logged as slow, and thus controls the extent of logging. Beginning with version 5.1.21, you can have fine-grained control by specifying the threshold even in fractions of seconds! To effect the change, restart the MySQL instance, as follows:

```
[mysqld]
slow_query_log=/tmp/slow_queries.log
long_query_time=1
```

Note: It is best you refer to the MySQL manual to determine the exact names of these variables, for your version of MySQL. Additionally, ensure that the destination folder for the log file already exists.

3. **Determine candidate queries for tuning:** If you examine a slow query log (see the sample below), the following information is displayed for each query that is logged:
   - Query execution time
   - Lock time
   - Total rows examined
   - Total rows sent

```
# Time: 101019 7:51:18
# User@Host: root[root] @ localhost []
#Query_time: 5.788721 Lock_time: 0.002104 Rows_sent: 52748 Rows_
examined: 52748
SET timestamp=1287489078;
```

Given that these logs are very long, you will need to summarise the information in the log. For this, we recommend that you use the *mysqldumpslow* utility that comes packaged with MySQL. It works on the slow query log file, and summarises the results, grouping queries based on similarity, and excluding differences in the values of number or string data. The command syntax is *mysqldumpslow [options] [log_file …]*. For details on command-line options, please refer to [1].

Let's now summarise the log, extracting the top 10 slow queries based on total execution time, which is the product of average query execution time, and the number of times the query was logged, as shown below:

```
mysqldumpslow -s t -t 10 /tmp/slowquery.log

Few lines from the output
Reading mysql slow query log from tmp/slow_queries.log
Count: 998  Time=4.90s (4893s)  Lock=0.12s (123s)  Rows=52748.0
(52642504), root[root]@localhost
```

In the above, *-s* defines the sort order, and sorting can happen with six different parameters. Here we choose *t* to specify the total execution time. Further, you should also sort on the following bases:

- *at: average query time*, to identify queries purely based on high query time.
- *l: lock time*, to identify queries that have serious contentions or possibility of deadlocks.
- *r: number of records output*, to tune at the single-request level the number of records fetched.

In our example, we listed the worst-performing queries in terms of their execution time, and stored the output in a file named *myworst10.txt*, which can be used for subsequent analysis.

Fix them individually

Now that you have a list of the top 10 queries to be optimised, the next step is to tune these queries individually. One part of this task is to run the queries through a profiler that will list the break-up of query execution time, and indicate the hot spots. The second part is fixing this. Here we will focus on the first part, as the methods used to fix queries is a separate subject in itself, and there are plenty of valuable references on the topic, such as [2].

For profiling, we recommend *mk-query-profiler*, one of the most versatile tools provided by *maatkit.org* [3]. *mk-query-profiler* can read, as input, the file we created in the earlier step, *myworst10.txt*, and execute the queries in it with analysis, for output as shown in the example below (for one query):

```
+--------------------------------------------------------+
|                    1 (2.4254 sec)                      |
+--------------------------------------------------------+
__ Overall stats _____________________ Value _____________
   Total elapsed time                          2.425
   Questions                                       1
     COMMIT                                        0
     DELETE                                        0
     DELETE MULTI                                  0
     INSERT                                        0
     INSERT SELECT                                 0
     REPLACE                                       0
     REPLACE SELECT                                0
     SELECT                                        1
     UPDATE                                        0
     UPDATE MULTI                                  0
   Data into server                               60
   Data out of server                        2080902
   Optimizer cost                           11042.127

__ Table and index accesses ___________ Value _____________
   Table locks acquired                            1
   Table scans                                     1
```